Humera Nuzhat

2303A52083

Batch 39

AI ASSISTED CODING-9.2 ASSIGNMENT

```python
# Task Description -1 (Documentation - Function Summary
# Generation)
# Task:
# Use AI to generate concise functional summaries for each Python
# function in a given script.
# Instructions:
# • Provide a Python script to the AI.
# • Ask the AI to write a short summary describing the purpose of
# each function.
# • Ensure summaries are brief and technically accurate.
# • Do not include code implementation details.
# Expected Output -1:
# A Python script where each function contains a clear and concise
# summary explaining its purpose.
# PROMPT: Write a short and technically accurate summary for each function in the given Python script.
#  Do not include implementation details. Keep the summaries concise and clear.
def calculate_average(numbers):
    total = sum(numbers)
    return total / len(numbers)
def find_maximum(numbers):
    return max(numbers)
def is_even(number):
    return number % 2 == 0

```

```python
# Task Description -2 (Documentation – Logical Explanation for
# Conditions and Loops)
# Task:
# Use AI to document the logic behind conditional statements and loops in
# a Python program.
# Instructions:
# • Provide a Python program without comments.
# • Instruct AI to explain only decision-making logic and loop
# behavior.
# • Skip basic syntax explanations.
# Expected Output -2:
# Python code with clear explanations describing the logic of conditions
# and loops.
# PROMPT:Explain only the decision-making logic and loop behavior in the following Python program.
#  Skip basic syntax explanations and focus only on conditional and loop logic
def process_scores(scores):
    passed = 0
    failed = 0
    # Iterates through each score in the list to evaluate results.
    for score in scores:
        # Checks whether the score meets the passing threshold (50).
        # If true, it increments the passed counter.
        if score >= 50:
            passed += 1
        # If the score is below 50, it increments the failed counter.
        else:
            failed += 1
    return passed, failed
```

```python
# Task Description -3 (Documentation – File-Level Overview)
# Task:
# Use AI to generate a high-level overview describing the functionality of
# an entire Python file.
# Instructions:
# • Provide the complete Python file to AI.
# • Ask AI to write a brief overview summarizing the file’s purpose
# and functionality.
# • Place the overview at the top of the file.
# Expected Output -3:
# A Python file with a clear and concise file-level overview at the
# beginning.
# Write a brief and clear file-level overview summarizing the purpose and functionality of this Python file.
# Place the overview at the top of the file
def add(a, b):
    return a + b
def subtract(a, b):
    return a - b
def multiply(a, b):
    return a * b
def divide(a, b):
    if b != 0:
        return a / b
    return None
# Task Description -4 (Documentation – Refine Existing
```

```python
# Task Description -4 (Documentation - Refine Existing
# Documentation)
# Task:
# Use AI to improve clarity and consistency of existing documentation in
# Python code.
# Instructions:
# • Provide Python code containing basic or unclear comments.
# • Ask AI to rewrite the documentation to improve clarity and
# consistency.
# • Ensure technical meaning remains unchanged.
# Expected Output -4:
# Python code with refined and improved documentation that is clear and
# consistent.
# PROMPT:Improve the clarity and consistency of the existing documentation in the following Python code.
#  Do not change the technical meaning
def calculate_discount(price, discount):
    # this function calculates something
    # discount is applied
    return price - (price * discount / 100)
```

```python
# Task Description -5 (Documentation - Prompt Detail Impact Study)
# Task:
# Study the impact of prompt detail on AI-generated documentation
# quality.
# Instructions:
# Create two prompts: one brief and one detailed.
# • Use both prompts to document the same Python function.
# • Compare the generated outputs.
# Expected Output -5:
# A comparison table highlighting differences in completeness, clarity, and
# accuracy of documentation.
# PROMPT:Write clear, structured documentation for this function. Include purpose, parameter descriptions with types, return value, and ensure technical accuracy. Do not include implementation details.
def calculate_interest(principal, rate, time):
    return (principal * rate * time) / 100

print(calculate_average([10, 20, 30]))
print(find_maximum([5, 8, 2]))
print(is_even(4))
'''
```

ANALYSIS:

Task 1 – Function Summary Generation

AI effectively generates concise and technically accurate summaries that describe each function's purpose without exposing implementation details.
This improves code readability and maintainability.

Task 2 – Logical Explanation for Conditions and Loops

AI clearly explains the decision-making logic and loop behavior while avoiding unnecessary syntax details.
This enhances understanding of program flow and conditional execution.

Task 3 – File-Level Overview

AI provides a high-level summary that describes the overall purpose and functionality of the entire file.
This helps readers quickly understand the module before reviewing the code.

Task 4 – Refine Existing Documentation

AI improves clarity, structure, and consistency of comments without changing the technical meaning.
The refined documentation appears more professional and easier to understand.

Task 5 – Prompt Detail Impact Study

Detailed prompts produce more structured, complete, and accurate documentation compared to brief prompts.
This demonstrates that prompt specificity significantly impacts AI output quality